

Practical-1

CODETANTRA

Home

Learn Anywhere

202501050029@mitaoe.ac.inSupportLogout

1.1.1. Calculate Momentum

10:10

AA

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

- m is the mass of the object (in kilograms).
- v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

calculate...

1#Type Content here...
2mass= float(input())
3velocity = float(input())
4momentum = mass* velocity
5print(f"momentum:.2f)kgm/s")

TerminalTest cases

< Prev

Reset

Submit

Next >

1.1.2. Conditional Calculation Based on the Number of Digits

06:32

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

- $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer n .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

condition...

Submit

Debugger

```

1 #write your code here...
2 import math
3 n= int(input())
4 if 0<=n and n<=9:
5     print(n*n)
6 elif 10<=n and n<=99:
7     print(f"{math.sqrt(n):.2f}")
8 elif 100<=n and n<=999:
9     print(f"{math.cbrt(n):.2f}")
10 else:
11     print("Invalid")

```

Terminal Test cases

1.1.3. Days between Two Dates

18:20

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format $YYYY-MM-DD$.
- The second line contains the second date in the format $YYYY-MM-DD$.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

noOfDay...

Submit

Debugger

```

1 from datetime import date
2
3 from datetime import datetime
4
5 date1= input().strip()
6 date2= input().strip()
7
8 d1 = datetime.strptime(date1, "%Y-%m-%d")
9 d2= datetime.strptime(date2, "%Y-%m-%d")
10
11 difference = d2 - d1
12
13 print(difference.days)
14 #write your code here...

```

Terminal Test cases

1.1.4. Reverse a Number

03:25

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases

reverseN...

Submit

Debugger

```

1 #write your code here...
2 n= int(input())
3 reversed_num = int(str(n)[::-1])
4 print(reversed_num)

```

Terminal Test cases

1.1.5. Multiplication Table

02:07

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

+

Explorer

1

#write your code here...

2

n = int(input())

3

for i in range(1,11):

4

print(f"{n} x {i} = {n*i}")

5

6

Terminal

Test cases

Submit

Debugger

1.2.1. Pass or Fail

03:03

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Explorer

1

n = int(input())

2

marks = list(map(int, input().split()))

3

failed = False

4

for m in marks:

5

if m < 40:

6

failed = True

7

break

8

9

if failed:

10

print("Fail")

11

else:

12

total = sum(marks)

13

agg = total / n

14

15

print("Aggregate Percentage: "f{ agg:.2f}")

16

if agg > 75:

17

print("Grade: Distinction")

18

elif(agg >= 60):

19

print("Grade: First Division")

20

elif(agg >= 50):

21

print("Grade: Second Division")

22

elif(agg >= 40):

23

print("Grade: Third Division")

24

Terminal

Test cases

Submit

Debugger

1.2.2. Fibonacci Series

11:15

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases

+

Explorer

1

def fibonacci(n):

2

3

if n == 1:

4

return 0

5

6

elif n == 2:

7

return 1

8

else:

9

return fibonacci(n - 1) + fibonacci(n - 2)

10

11

n = int(input())

12

for i in range(1, n + 1):

13

print(fibonacci(i), end=" ")

Terminal

Test cases

Submit

Debugger

1.2.3. Pattern - 1 15/52

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

rightangl...

```
1 #Type Content here...
2 n = int(input())
3 for i in range(1,n+1):
4     print('*'* i)
5
```

Terminal Test cases

Submit

1.2.4. Pattern - 2 15/52

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases +

numberP...

```
1 # Type Content here...
2 n= int(input())
3 for i in range(1, n+1):
4     for j in range(1, i+1):
5         print(j, end=" ")
6     print()
7
```

Terminal Test cases

Submit